



Multi2sim Kepler: A Detailed Architectural GPU Simulator

Xun Gong, Rafael Ubal, David Kaeli

Northeastern University Computer Architecture Research Lab

Department of Electrical and Computer Engineering

Northeastern University

Boston, MA

WHY USE SIMULATORS



Northeastern University

- Designing and fabricating chips are expensive
 - A significant amount of the cost of delivering a new chip involves design verification/validation
 - May take many years to fully test a new microarchitecture
 - Challenging to predict the performance and power prior to silicon
- Leverage software to evaluate models of proposed designs
 - Support design space exploration
 - Allows validation before hardware becomes available
 - Allows software developers to evaluate optimize performance

BACKGROUND



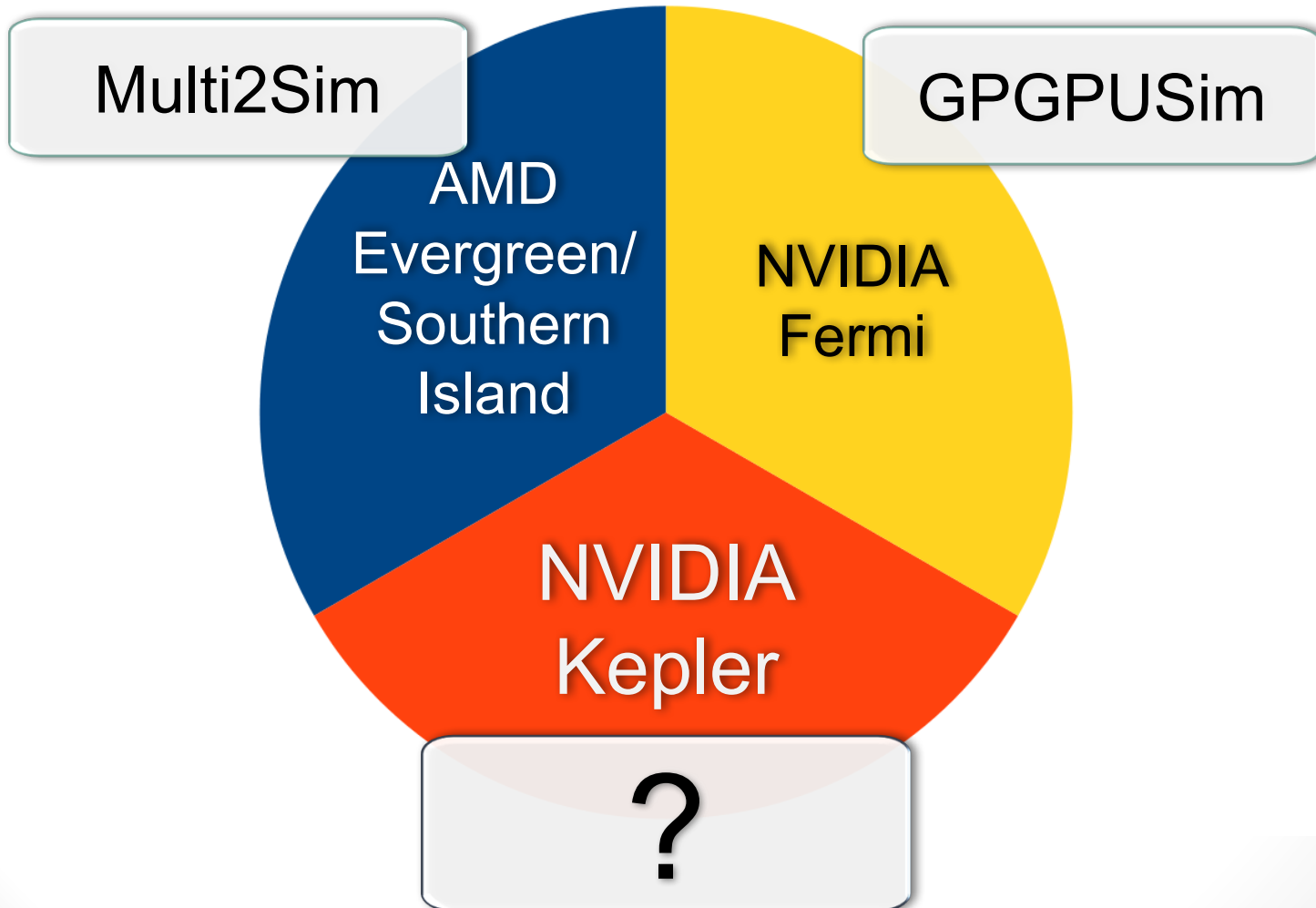
Northeastern University

- GPU has become pervasive in high performance and data center environments
- Simulation is one of the key toolsets for computer architects to evaluate future designs
- Given the rapid growth in GPU computing, the research community requires accurate GPU simulation tools

BACKGROUND



Northeastern University





INTRODUCTION

MULTI2SIM SIMULATION FRAMEWORK

- A simulator for CPU, GPU and Heterogeneous systems
 - Support for CPU architectures: X86, ARM, and MIPS
 - Support for GPU architectures: AMD southern islands, NVIDIA Kepler
 - Support for HSA Intermediate Language
- Based on C++ 11
- Large user base and open source developer community
- Maintained through Github (<https://github.com/multi2sim>)



INTRODUCTION

MULTI2SIM SIMULATION FRAMEWORK

	Disasm.	Emulation	Timing Simulation	Visual tool
ARM	✓	In progress	—	—
MIPS	✓	In progress	—	—
x86	✓	✓	✓	✓
AMD Southern Islands	✓	✓	✓	✓
NVIDIA Kepler	✓	✓	✓	In progress
HSA Intermediate Language	✓	✓	In progress	In progress

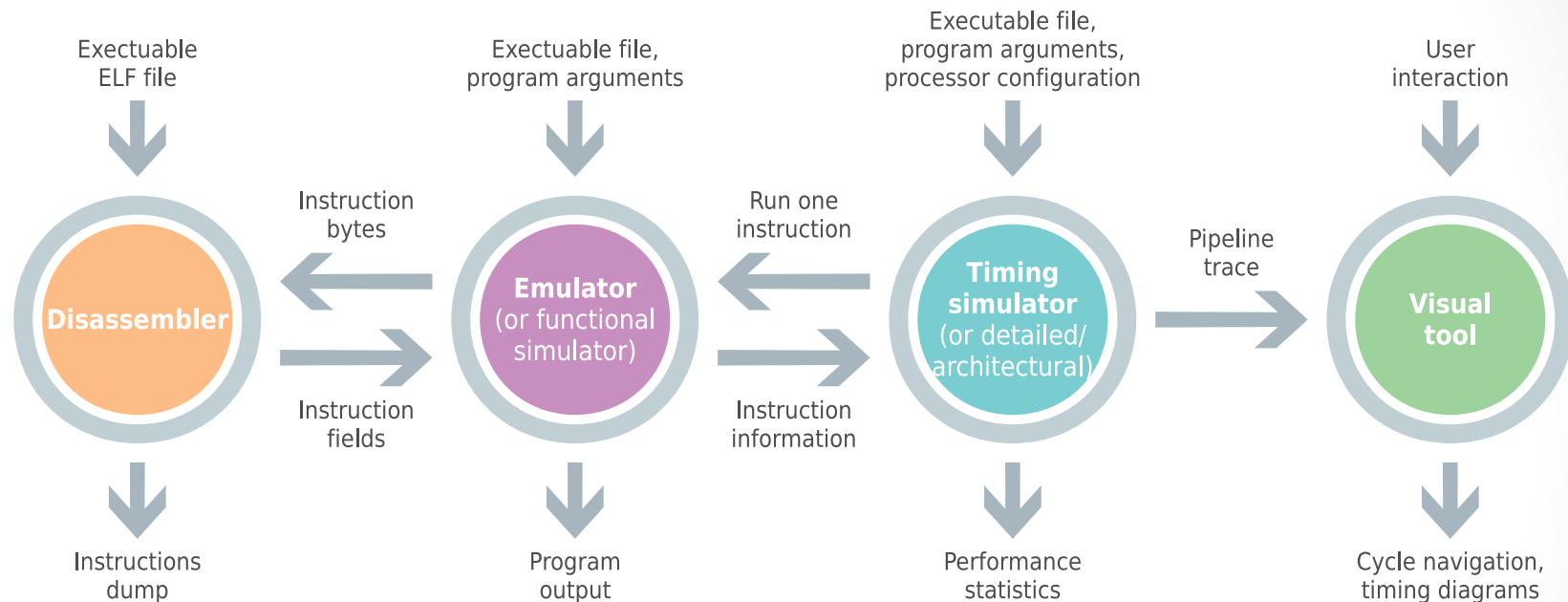
- Available in Multi2Sim 5.0

- NVIDIA Kepler, Southern Islands, and x86 supported
- Three other CPU/GPU architectures in progress



INTRODUCTION

MULTI2SIM SIMULATION FRAMEWORK



- **Modular implementation**

- Four clearly different software modules per architecture (x86, MIPS, Kepler....)
- Each module provides a standard interface for stand-alone execution, or interaction with other modules



Outline

- Introduction & Background
- CUDA Execution
- Kepler simulation
- Evaluation
- Conclusions

CUDA EXECUTION SIMULATION LEVEL



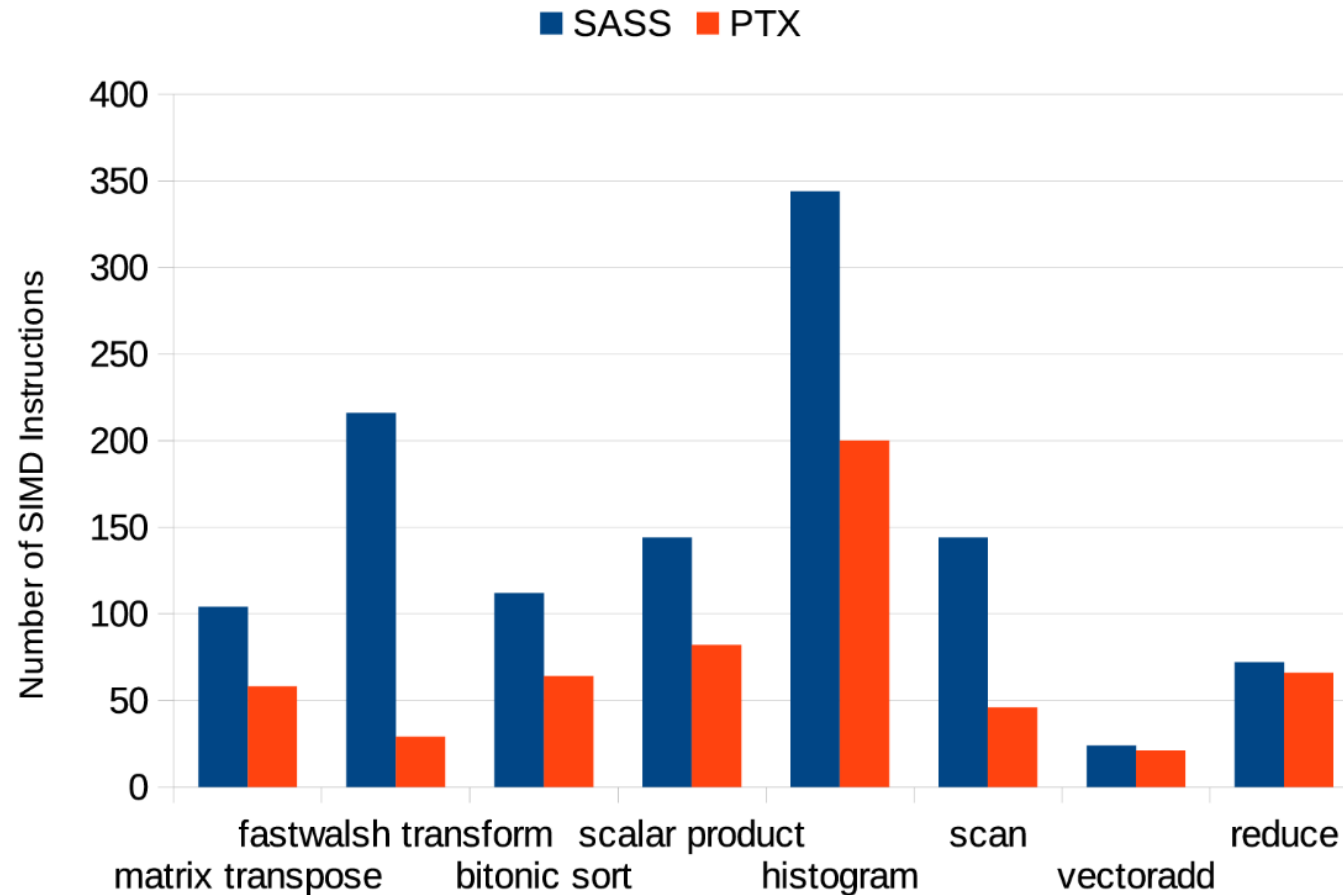
Northeastern University

- SASS: NVIDIA Shader Assembly, the native GPU ISA
 - PTX: a higher-level intermediate language compared to SASS defined by NVIDIA
 - The SASS code changes for each different generation of NVIDIA GPU, while PTX code is architecture independent
-
- ✓ Multi2Sim Kepler is designed to support NVIDIA SASS

CUDA EXECUTION SIMULATION LEVEL



Northeastern University



☹️ PTX execution is very different than SASS execution ☹️

CUDA EXECUTION SIMULATION LEVEL



Northeastern University

- It is important to run SASS
- The number of registers is limited in SASS, but is unlimited in PTX
- Schedulers will have more restrictions when working at the SASS level
- More ISA-specific issues can be considered when we run SASS
- Running SASS simulation is much closer to the actual execution in recent GPUs (i.e., Kepler GPUs)

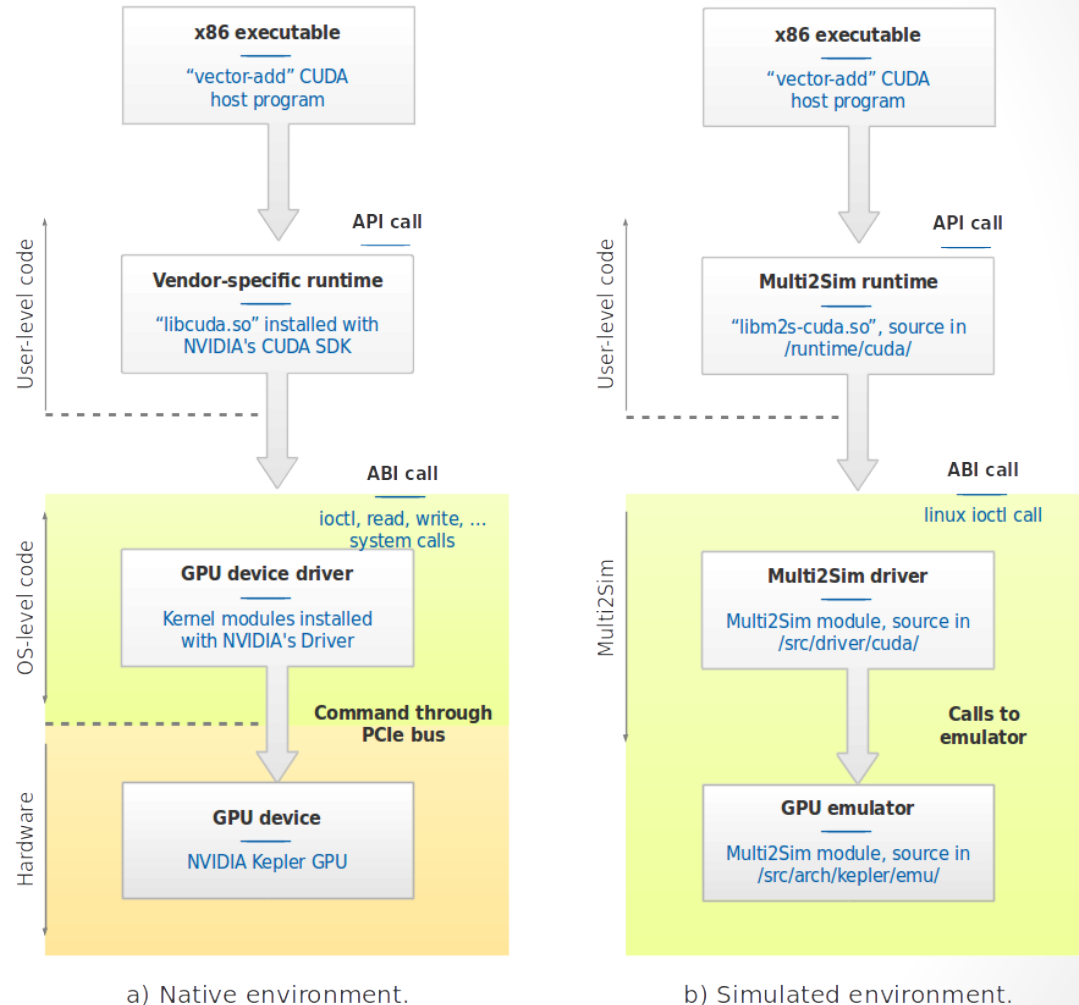
CUDA EXECUTION

CUDA SUPPORT ON MULTI2SIM



Northeastern University

- The figure shows the **modular organization** of the CUDA execution framework, based on 4 software/hardware entities.
- In each case, we compare **native execution** with **simulated execution**.



CUDA EXECUTION SIMULATION CHALLENGES



Northeastern University

- **Driver & Runtime APIs**

- Implement our own CUDA Driver & Runtime APIs

- **ISA Level**

- Reverse Engineering of the whole Kepler ISA since there is no public information

- **Microarchitecture**

- Implement benchmarks to reverse engineer and test all hardware related specifications



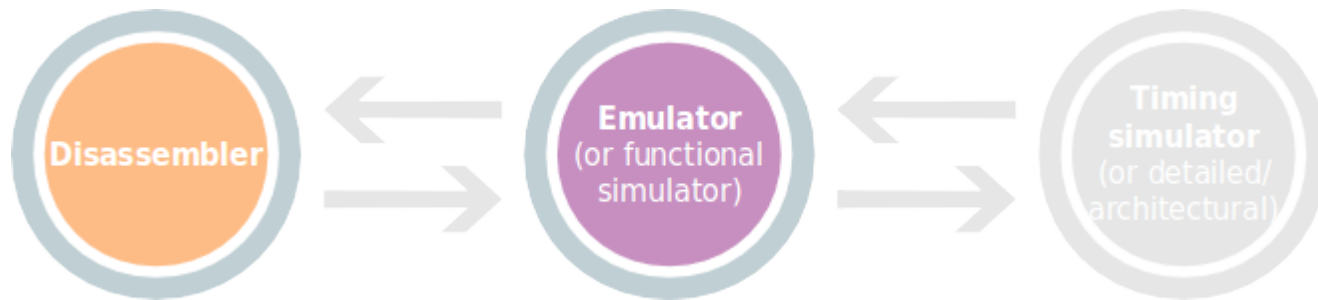
Outline

- Introduction & Background
- CUDA support on Multi2Sim
- Kepler simulation
- Evaluation
- Conclusions

KEPLER SIMULATION DISASSEMBLER & EMULATOR



Northeastern University



KEPLER SIMULATION

DISASSEMBLER & EMULATOR



Northeastern University

- **Disassembler**

- Reads from CUDA binary file and dumps a text-based output of all fragments of GPU ISA code found in the file
- Outputs SASS (shader assembly) instructions one by one to emulator

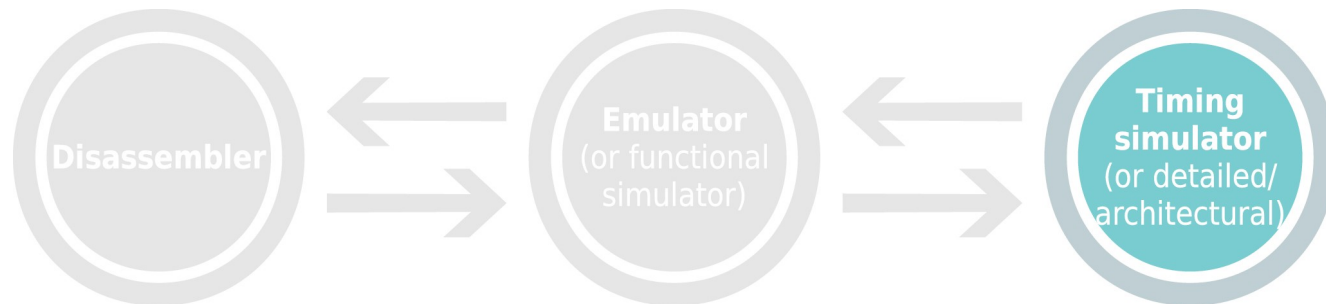
- **Emulator**

- Reads instructions from disassembler, reproduce the original behavior of a guest program
- Providing instructions information to timing simulator
- Support CUDA SDK 6.5 benchmark suite (21 supported), other benchmark suite will be supported in the future

KEPLER SIMULATION TIMING SIMULATOR



Northeastern University

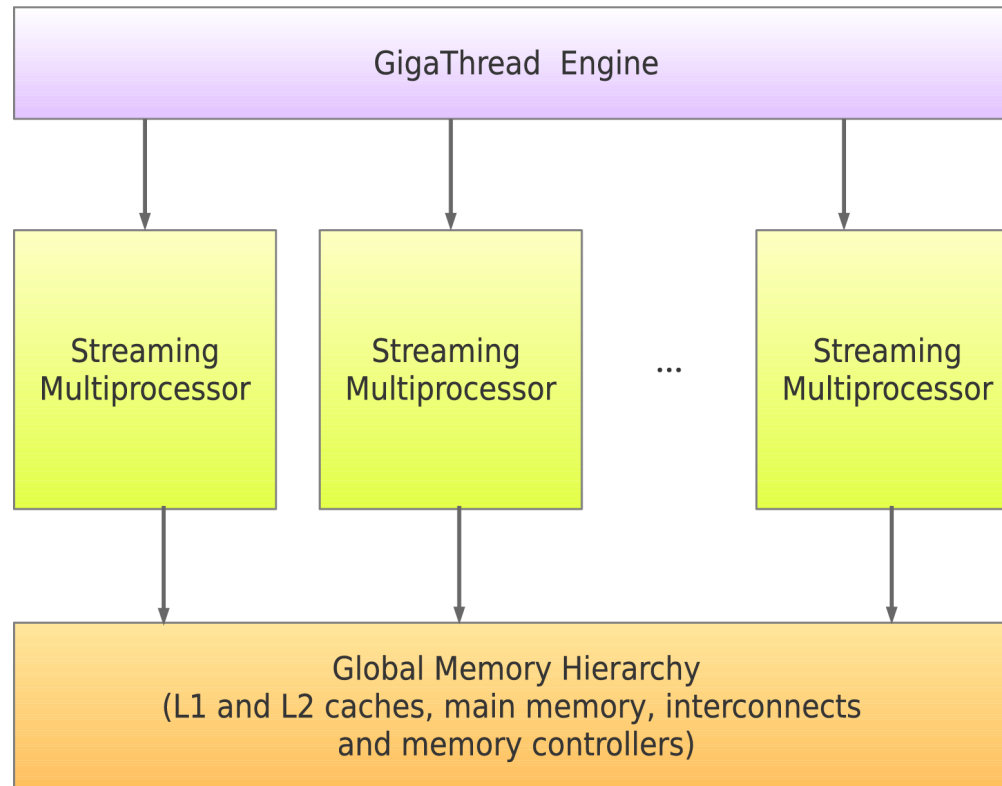


KEPLER SIMULATION

TIMING SIMULATION



Northeastern University

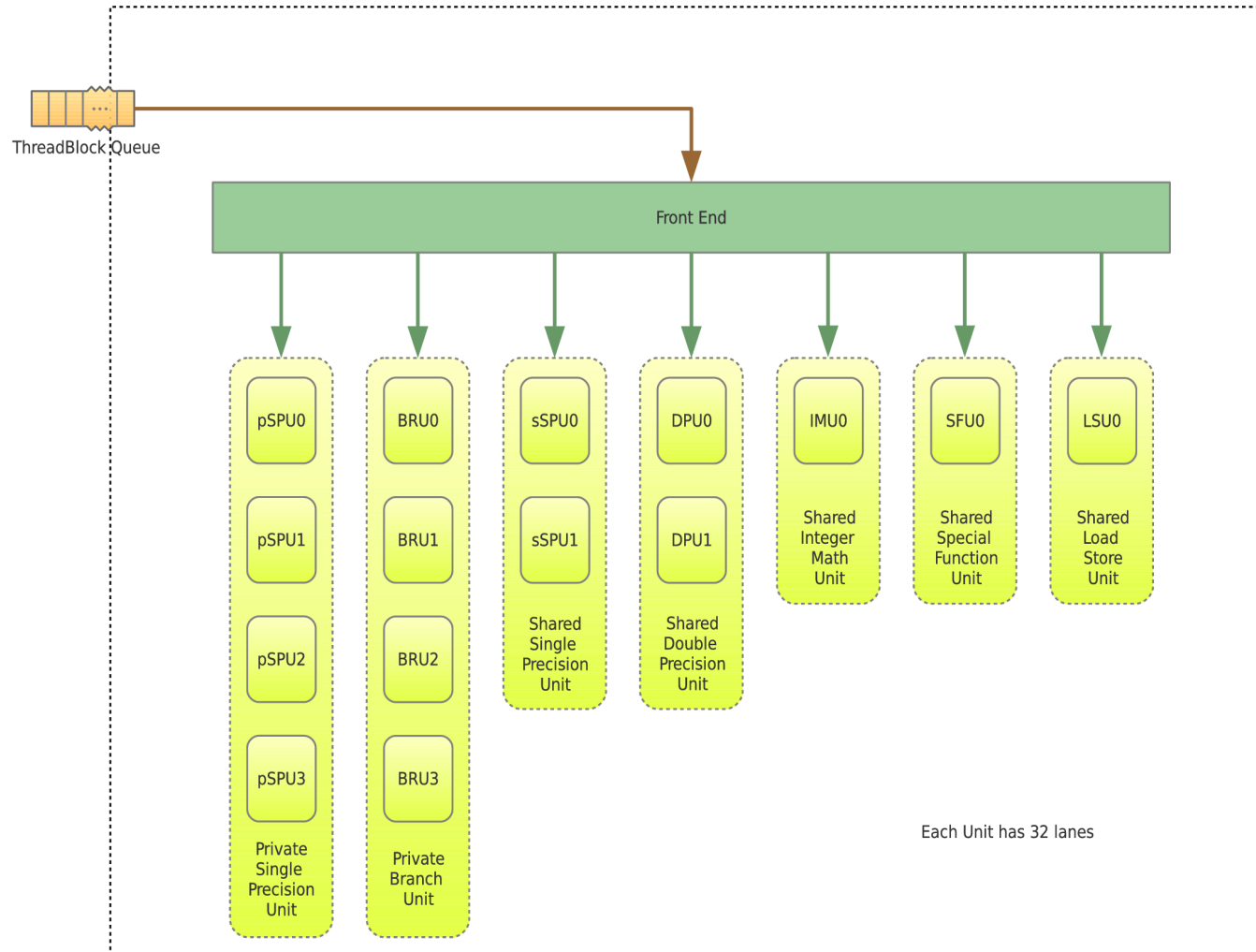


KEPLER SIMULATION

TIMING SIMULATION



Northeastern University



KEPLER SIMULATION

TIMING SIMULATION



Northeastern University

- Support for detailed architectural models for GPU hardware components
 - SMs, Warp schedulers, execution units, memory and etc.
- Support for instruction pipeline exploration
 - Pipelines for different kinds of instructions such as integer, floating point and control flow
- Provides architecture-related statistics
 - Cache miss/hits, instructions retired, occupancy, etc.

KEPLER SIMULATION EMULATOR



Northeastern University

- Produces CUDA kernel results
 - Emulates instructions and updates registers and memory
- Produces execution statistics
 - Number of executed grids and blocks
 - Dynamic instruction mix of the kernel and etc.
- Produces an ISA-level trace
 - Instruction emulation trace

KEPLER SIMULATION

ARCHITECTURAL SIMULATION



Northeastern University

- Models SMs, memory hierarchy and other hardware details
- Maps thread blocks onto SMs and warp pools
- Emulates instructions and propagates state through the execution pipelines
 - Models resource usage and contention

KEPLER SIMULATION

MULTI2SIM KEPLER ADVANTAGES



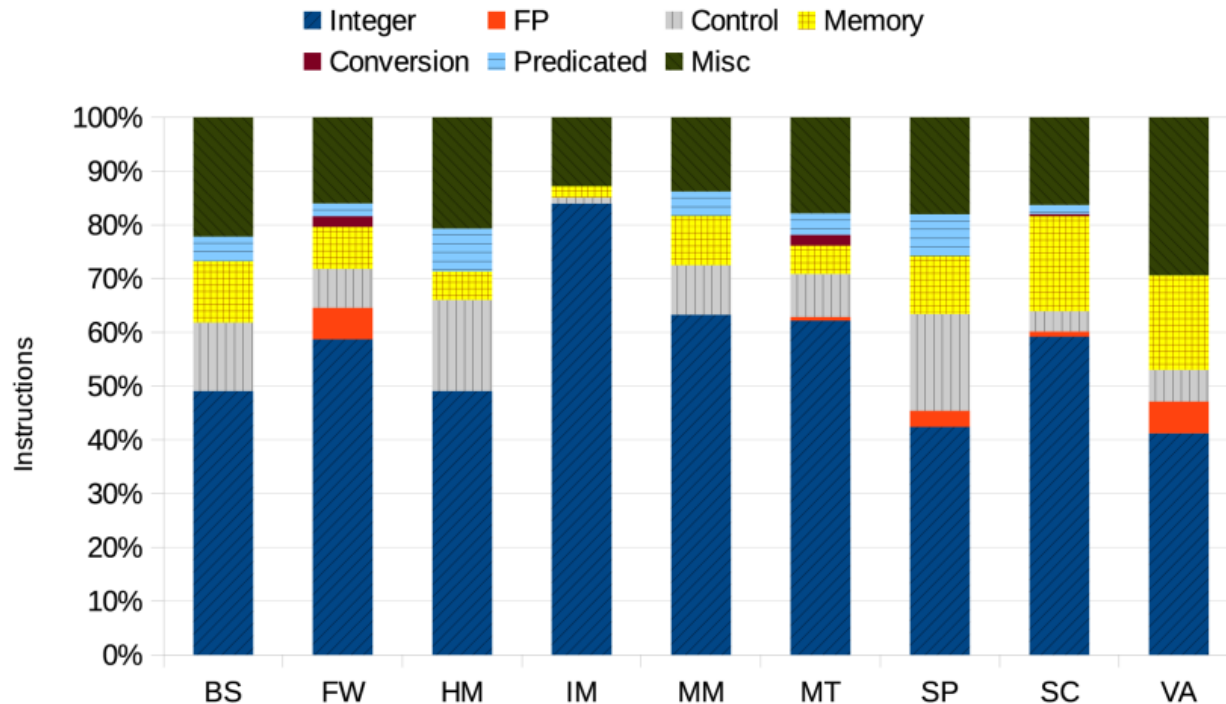
Northeastern University

- Support for CPU-GPU heterogeneous simulation
- Support for NVIDIA Kepler native SASS execution
- Support for detailed NVIDIA Kepler microrarchitectural exploration



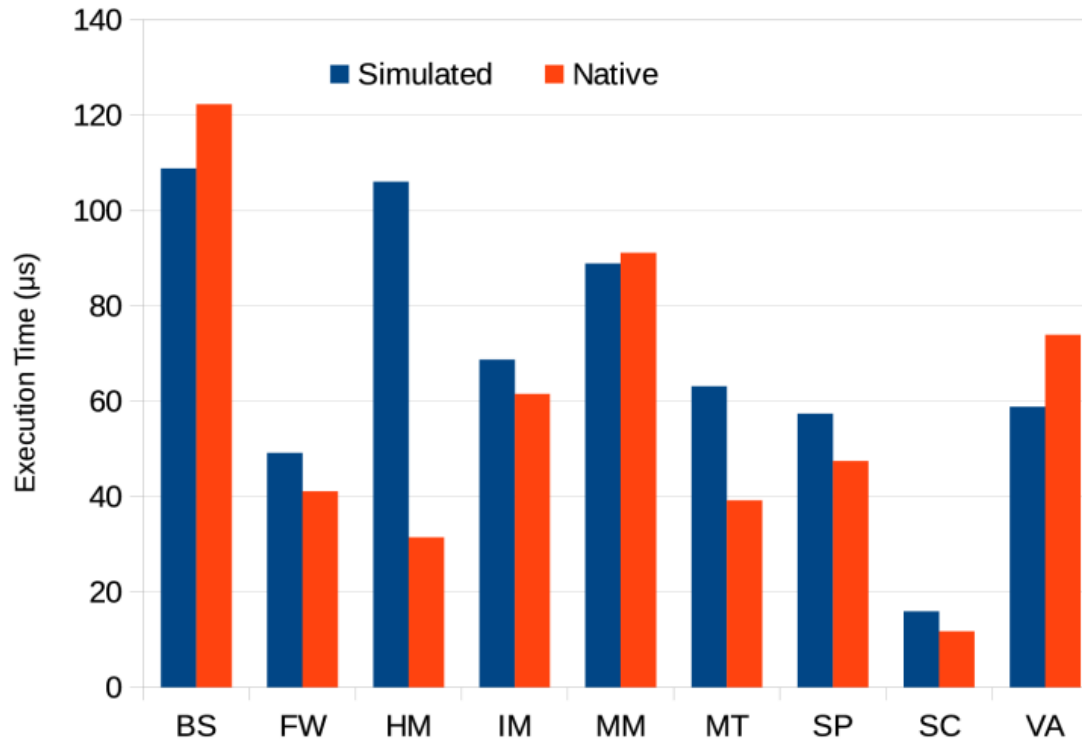
Outline

- Introduction & Background
- CUDA support on Multi2Sim
- Kepler simulation
- Evaluation
- Conclusions

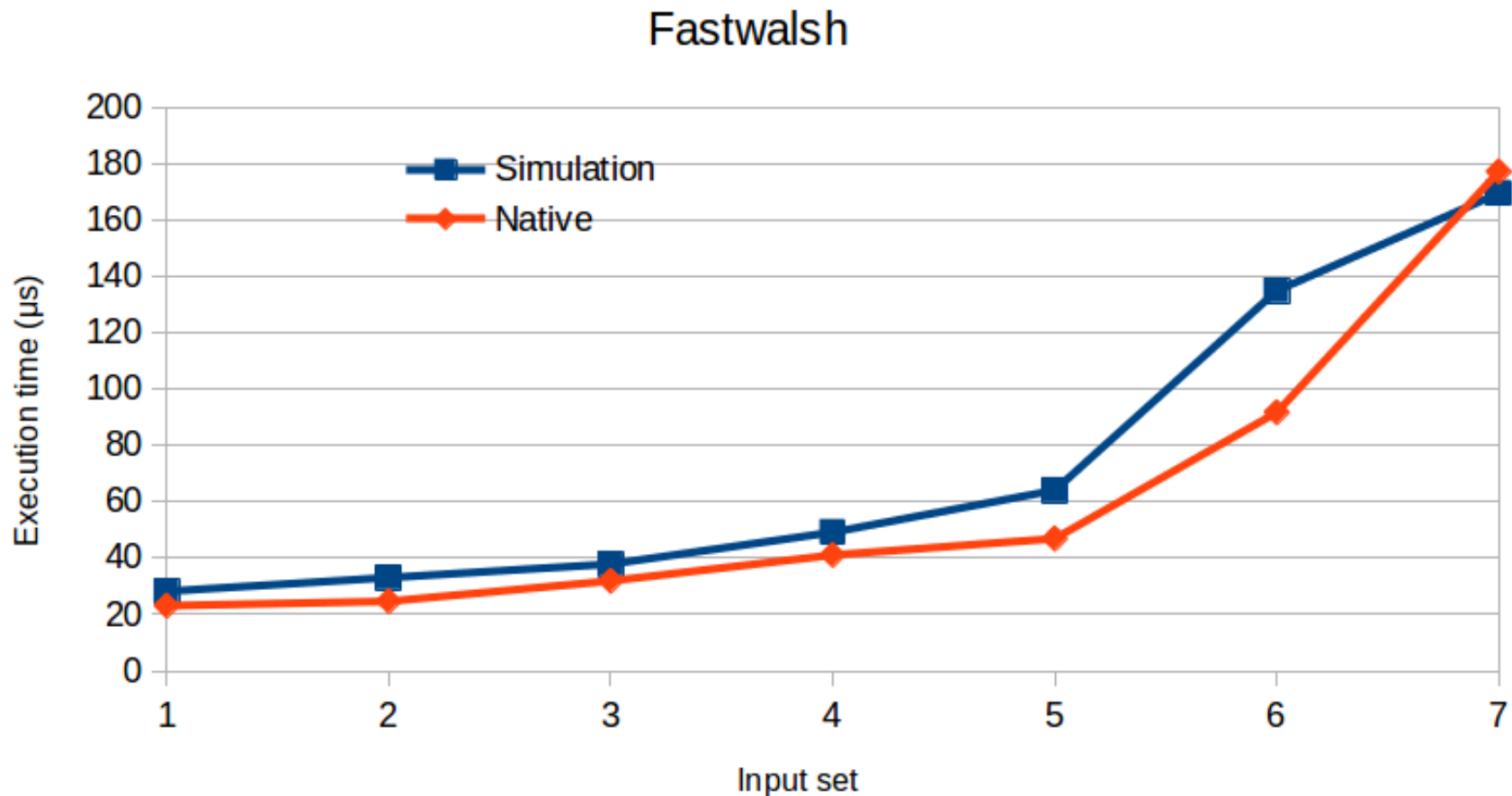


- Emulator
 - Statistics: Number of instructions executed, instructions classification, percentage of each kind instruction

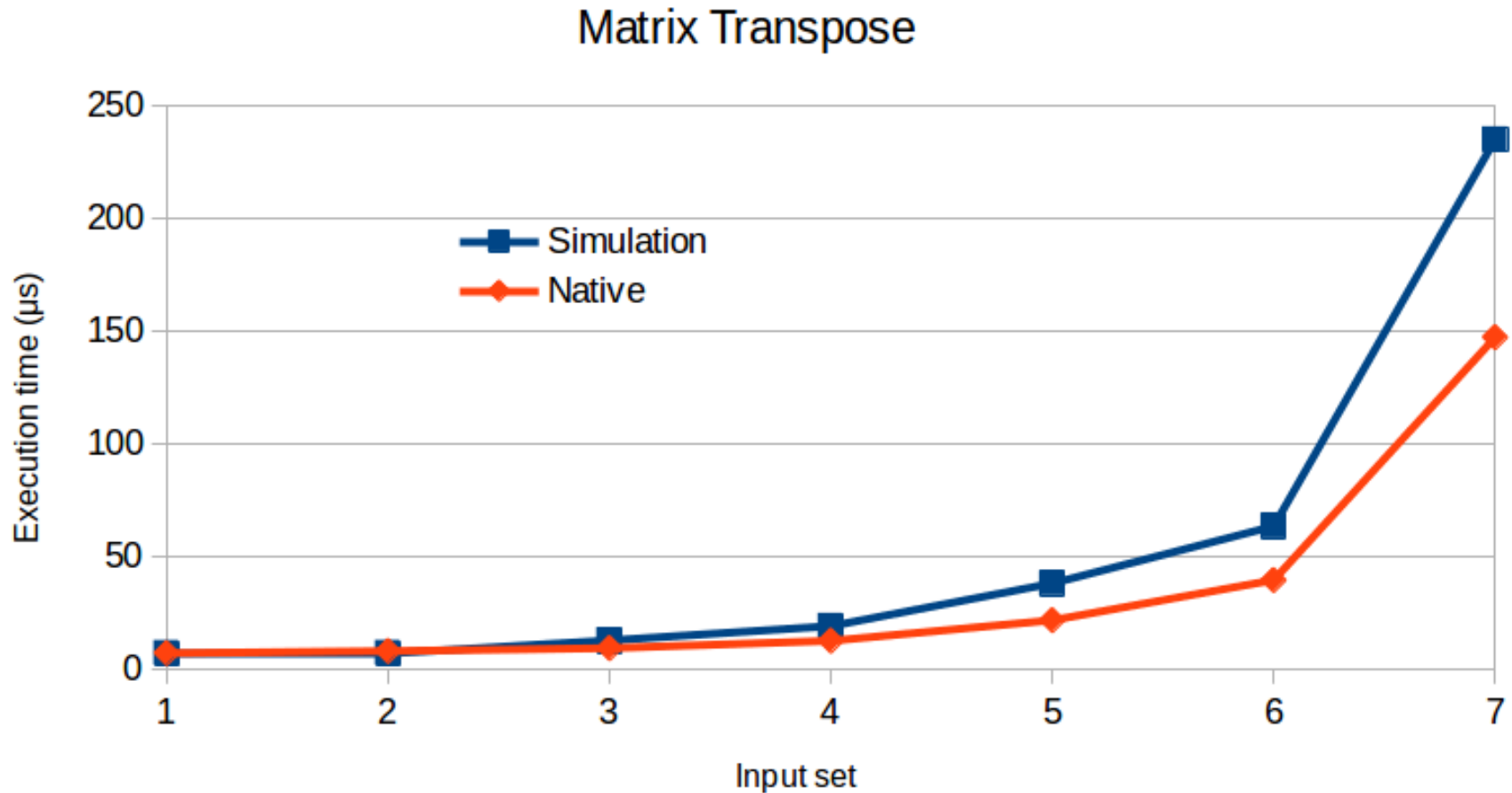
EVALUATION



- Average execution time for different input sets on each benchmark
- In general, there is good fidelity with the K20X
- HM is an outlier, since it uses `st.wt` and `ld.cv` instructions, changing cache policy



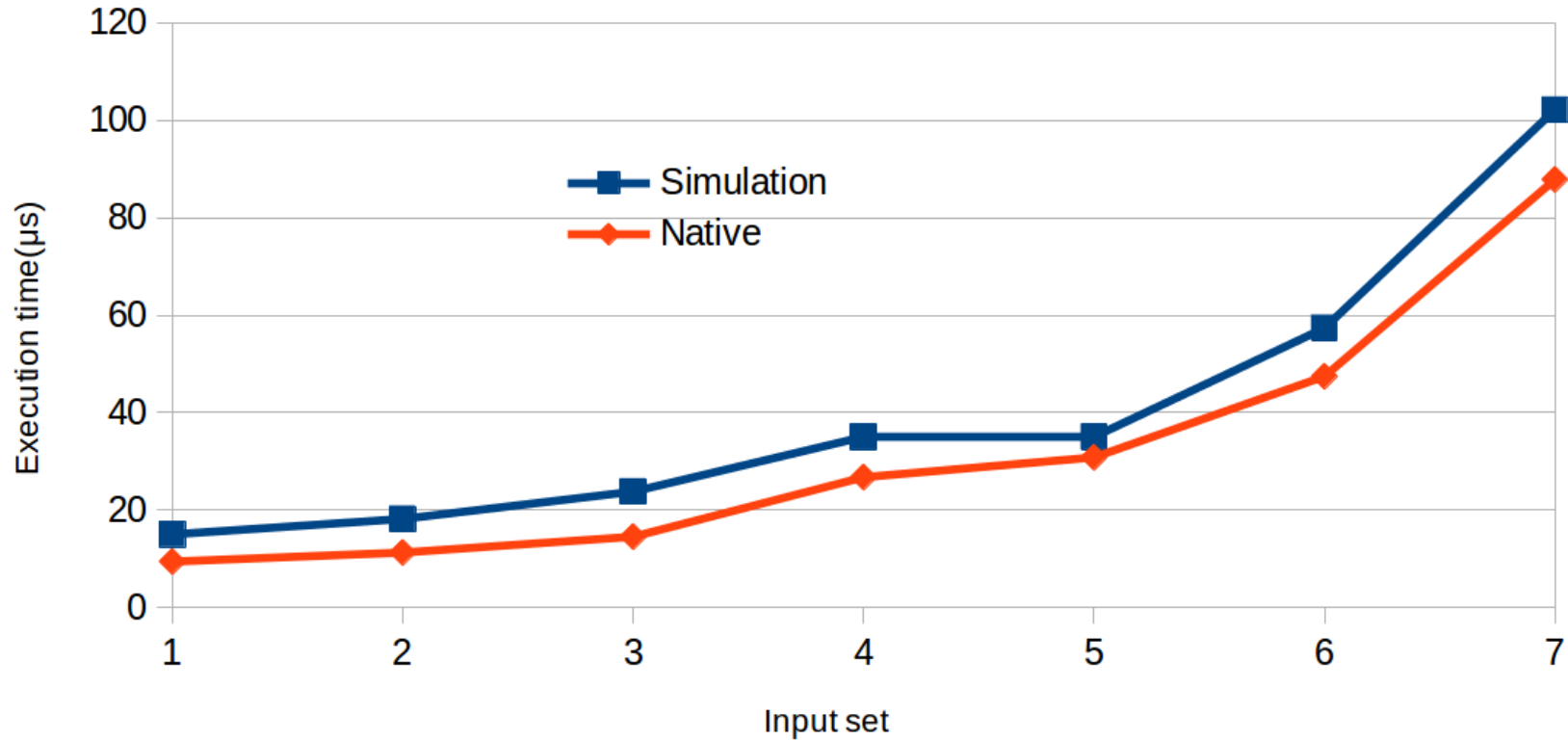
- Input sizes: From 1K to 128K



- Input size: From 128x128, to 1024x1024



Scalar Product

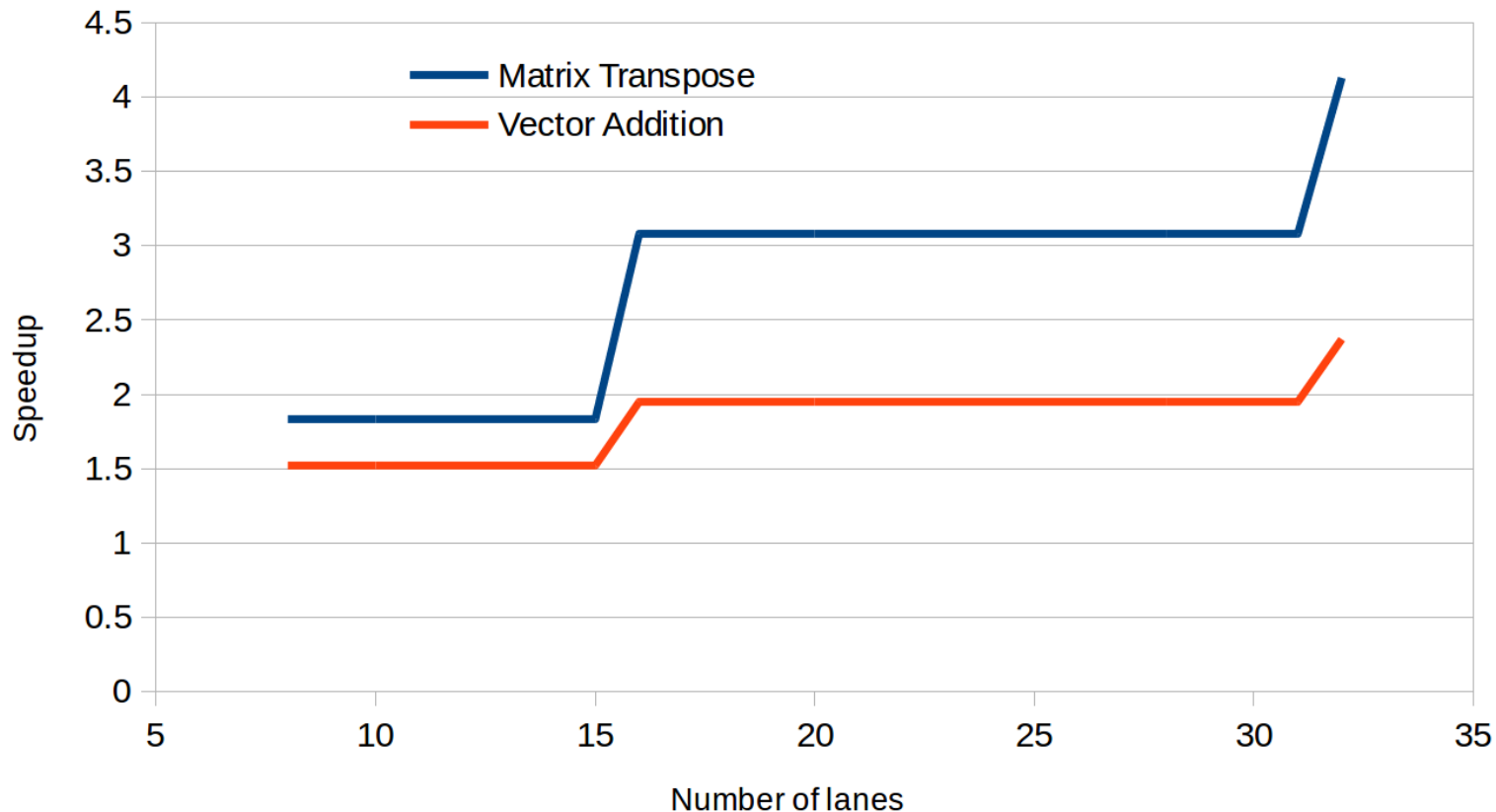


- Input sizes: From 32K to 1M

EVALUATION



Northeastern University



- Performance achieved by changing the number of lanes for each pSPU per SMX
- MatrixTranspose shows greater speedup than VectorAdd, because it is less memory sensitive



Outline

- Introduction & Background
- CUDA support on Multi2Sim
- Kepler simulation
- Evaluation
- Conclusions



- **Summary**

- Presented Multi2sim Kepler, a detailed performance simulator supporting NVIDIA Kepler SASS execution
- Provided example architectural studies, exploring Kepler GPU microarchitecture
- Showed the benefits of the infrastructure by evaluating application characteristics

- **Future work**

- Support more benchmarks
- Implement new CUDA runtime and driver APIs
- Improve the accuracy of our simulator, focusing on memory model



Thank you!

Questions?



* This work is supported in part by NSF Grant CNS-1525412, and through generous donations from NVIDIA, AMD and the Heterogeneous Systems Foundation.